

Automaton

Генериран от Doxygen 1.17.0

1 Класове Йерархичен указател	1
1.1 Класове Йерархия	1
2 Класове Указател	3
2.1 Класове Списък	3
3 Файлове Списък	5
3.1 Файлове Списък	5
4 Класове Документация	7
4.1 Automaton Клас Препратка	7
4.1.1 Подробно описание	8
4.1.2 Конструктор & Деструктор Документация	8
4.1.2.1 Automaton()	8
4.1.3 Членове Функции(методи) Документация	9
4.1.3.1 addTransition()	9
4.1.3.2 createState()	9
4.1.3.3 getAlphabet()	9
4.1.3.4 getReversed()	10
4.1.3.5 getStatesCount()	10
4.1.3.6 isDeterministic()	10
4.1.3.7 recognise()	10
4.1.4 Приятели и Свързани функции Документация	11
4.1.4.1 Concat	11
4.1.4.2 Star	11
4.1.4.3 Union	11
4.2 AutomatonCLI Клас Препратка	12
4.2.1 Подробно описание	12
4.3 AutomatonSerializer Клас Препратка	13
4.3.1 Членове Функции(методи) Документация	13
4.3.1.1 serialize()	13
4.4 BadSymbolException Клас Препратка	13
4.4.1 Подробно описание	14
4.5 ConcatRegEx Клас Препратка	14
4.5.1 Подробно описание	15
4.5.2 Конструктор & Деструктор Документация	15
4.5.2.1 ConcatRegEx()	15
4.5.3 Членове Функции(методи) Документация	15
4.5.3.1 clone()	15
4.5.3.2 eval()	15
4.5.3.3 toAutomaton()	15
4.5.3.4 toString()	16
4.6 Exception Клас Препратка	16
4.6.1 Подробно описание	16

4.7 LiteralRegEx Клас Препратка	16
4.7.1 Подробно описание	17
4.7.2 Членове Функции(методи) Документация	17
4.7.2.1 clone()	17
4.7.2.2 eval()	17
4.7.2.3 toAutomaton()	17
4.7.2.4 toString()	18
4.8 ParserException Клас Препратка	18
4.8.1 Подробно описание	18
4.8.2 Членове Функции(методи) Документация	19
4.8.2.1 toString()	19
4.9 RegEx Клас Препратка	19
4.9.1 Подробно описание	19
4.9.2 Членове Функции(методи) Документация	20
4.9.2.1 eval()	20
4.9.2.2 toAutomaton()	20
4.9.2.3 toString()	20
4.10 RegExParser Клас Препратка	20
4.10.1 Подробно описание	21
4.10.2 Членове Функции(методи) Документация	21
4.10.2.1 parse()	21
4.11 StarRegEx Клас Препратка	21
4.11.1 Подробно описание	22
4.11.2 Членове Функции(методи) Документация	22
4.11.2.1 clone()	22
4.11.2.2 eval()	22
4.11.2.3 toAutomaton()	22
4.11.2.4 toString()	22
4.12 StateExistsException Клас Препратка	23
4.12.1 Подробно описание	23
4.12.2 Членове Функции(методи) Документация	23
4.12.2.1 toString()	23
4.13 Transition Структура Препратка	24
4.13.1 Подробно описание	24
4.14 UnionRegEx Клас Препратка	24
4.14.1 Подробно описание	25
4.14.2 Конструктор & Деструктор Документация	25
4.14.2.1 UnionRegEx()	25
4.14.3 Членове Функции(методи) Документация	25
4.14.3.1 clone()	25
4.14.3.2 eval()	25
4.14.3.3 toAutomaton()	25
4.14.3.4 toString()	26

5	Файлове Документация	27
5.1	Automaton.hpp	27
5.2	AutomatonSerializer.hpp	28
5.3	BadSymbolException.hpp	29
5.4	Exception.hpp	29
5.5	ParserException.hpp	29
5.6	StateExistsException.hpp	29
5.7	ConcatRegEx.hpp	30
5.8	LiteralRegEx.hpp	30
5.9	RegEx.hpp	30
5.10	RegExParser.hpp	30
5.11	StarRegEx.hpp	31
5.12	UnionRegEx.hpp	31
5.13	AutomatonCLI.hpp	31
	Азбучен указател	33

Глава 1

Класове Йерархичен указател

1.1 Класове Йерархия

Този списък с наследявания е сортиран, но не изцяло по азбучен ред:

Automaton	7
AutomatonCLI	12
AutomatonSerializer	13
Exception	16
BadSymbolException	13
ParserException	18
StateExistsException	23
Regex	19
ConcatRegex	14
LiteralRegex	16
StarRegex	21
UnionRegex	24
RegexParser	20
Transition	24

Глава 2

Класове Указател

2.1 Класове Списък

Класове, структури, обединения и интерфейси с кратко описание:

[Automaton](#)

Класът [Automaton](#) представлява краен автомат, който може да бъде недетерминиран (NFA) или детерминиран (DFA). Той съдържа състоянията и преходите между тях. Класът предоставя методи за създаване на нови състояния, добавяне на преходи, разпознаване на думи, обръщане на автомата, детерминизация и минимизация. Също така включва функции за обединение, конкатенация и звезда на автомати

7

[AutomatonCLI](#)

Класът [AutomatonCLI](#) предоставя интерфейс за команден ред, който позволява на потребителите да взаимодействат с автомати и регулярни изрази. Чрез този интерфейс потребителите могат да създават, модифицират, запазват и зареждат автомати, както и да извършват различни операции като детерминизация, минимизация, обединение, конкатенация и звезда. CLI-то също така поддържа разпознаване на думи от автоматите и визуализация на автоматите чрез DOT формат

12

[AutomatonSerializer](#)

13

[BadSymbolException](#)

Изключение, което се хвърля при опит за използване на символ, който не е част от азбуката на автомата. Съдържа информация за символа, който е бил използван и не е валиден

13

[ConcatRegex](#)

Класът [ConcatRegex](#) е конкатенация на два други регулярни изрази

14

[Exception](#)

Базов клас за всички изключения в проекта. Той съдържа съобщение за грешката и предоставя метод `toString()` за връщане на това съобщение

16

[LiteralRegex](#)

Класът [LiteralRegex](#) представлява регулярни изрази, които съответстват на конкретна дума

16

[ParserException](#)

Изключение, което се хвърля при срещане на синтактична грешка по време на парсане на регулярни изрази. Съдържа текст описващ грешката, индекса където е възникнала грешката, и самия израз, който е бил парсан

18

[Regex](#)

Абстрактен базов клас за представяне на регулярни изрази

19

[RegexParser](#)

Класът [RegexParser](#) използва рекурсивен парсер за обработка на различните оператори в регулярните изрази, като обединение (+), конкатенация(без оператор) и звезда (*). За празна дума се използва символът '@'

20

StarRegex

Класът **StarRegex** представлява регулярни изрази, които са звезда на друг регулярен израз 21

StateExistsException

Исключение, което се хвърля при опит за създаване на състояние с идентификатор, който вече съществува в автомата, или при опит за достъп до несъществуващо състояние. Съдържа информация за идентификатора на състоянието и дали то вече съществува или не 23

Transition

Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като '\0') 24

UnionRegex

Класът **UnionRegex** представлява обединение на два други регулярни изрази . . 24

Глава 3

Файлове Списък

3.1 Файлове Списък

Пълен списък с документираните файлове с кратко описание:

Automaton.hpp	27
AutomatonSerializer.hpp	28
BadSymbolException.hpp	29
Exception.hpp	29
ParserException.hpp	29
StateExistsException.hpp	29
ConcatRegEx.hpp	30
LiteralRegEx.hpp	30
RegEx.hpp	30
RegExParser.hpp	30
StarRegEx.hpp	31
UnionRegEx.hpp	31
AutomatonCLI.hpp	31

Глава 4

Класове Документация

4.1 Automaton Клас Препратка

Класът [Automaton](#) представлява краен автомат, който може да бъде недетерминиран (NFA) или детерминиран (DFA). Той съдържа състоянията и преходите между тях. Класът предоставя методи за създаване на нови състояния, добавяне на преходи, разпознаване на думи, обръщане на автомата, детерминизация и минимизация. Също така включва функции за обединение, конкатенация и звезда на автомати.

```
#include <Automaton.hpp>
```

Общодостъпни членове функции

- [Automaton](#) (const std::vector< char > alphabet)
Конструктор, който приема азбука от символи и създава празен автомат с тази азбука. Автоматът няма състояния или преходи при инициализация.
- void [createNewState](#) (size_t id, bool isStart, bool isFinal)
Създава ново състояние с даден идентификатор и го добавя към автомата. Може да се укаже дали състоянието е начално и/или крайно.
- void [addTransition](#) (size_t from, char symbol, size_t to)
Добавя преход от едно състояние към друго с даден символ. Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като ^0).
- bool [recognise](#) (const std::string &word) const
Проверява дали дадена дума се разпознава от автомата.
- [Automaton getReversed](#) () const
Връща нов автомат, който е обърнатата версия на текущия автомат. В обърнатия автомат всички преходи са обърнати, началните състояния стават крайни и крайни състояния стават начални.
- void [reverse](#) ()
Обръща текущия автомат. Всички преходи се обръщат, началните състояния стават крайни и крайни състояния стават начални.
- bool [isDeterministic](#) () const
Проверява дали автоматът е детерминиран. Автоматът е детерминиран, ако няма две прехода от едно и също състояние с един и същ символ и няма епсилон-преходи.
- [Automaton getDeterministic](#) () const
Връща нов автомат, който е детерминизираната версия на текущия автомат. Детерминизацията се извършва чрез алгоритъма за конвертиране на NFA в DFA.
- void [determinise](#) ()

- Детерминира текущия автомат.
- [Automaton](#) `getMinimised () const`
Връща нов автомат, който е минимизираната версия на текущия автомат. Минимизацията се извършва чрез алгоритъма на Бжозовски
- `void minimise ()`
Минимизира текущия автомат. Минимизацията се извършва чрез алгоритъма на Бжозовски
- `const std::vector< char > & getAlphabet () const`
- `size_t getStatesCount () const`

Приатели

- `class AutomatonSerializer`
- [Automaton Union](#) (`const Automaton &lhs, const Automaton &rhs`)
Функция за обединение на два автомата.
- [Automaton Concat](#) (`const Automaton &lhs, const Automaton &rhs`)
Функция за конкатенация на два автомата.
- [Automaton Star](#) (`const Automaton &automaton`)
Функция за звезда на Клини на автомат.

4.1.1 Подробно описание

Класът [Automaton](#) представлява краен автомат, който може да бъде недетерминиран (NFA) или детерминиран (DFA). Той съдържа състоянията и преходите между тях. Класът предоставя методи за създаване на нови състояния, добавяне на преходи, разпознаване на думи, обръщане на автомата, детерминизация и минимизация. Също така включва функции за обединение, конкатенация и звезда на автомати.

4.1.2 Конструктор & Деструктор Документация

4.1.2.1 Automaton()

```
Automaton::Automaton (
    const std::vector< char > alphabet)
```

Конструктор, който приема азбука от символи и създава празен автомат с тази азбука. Автоматът няма състояния или преходи при инициализация.

Аргументи

alphabet	Вектор от символи, представляващи азбуката на автомата.
----------	---

4.1.3 Членове Функции(методи) Документация

4.1.3.1 addTransition()

```
void Automaton::addTransition (
    size_t from,
    char symbol,
    size_t to)
```

Добавя преход от едно състояние към друго с даден символ. Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като '\0').

Аргументи

from	Идентификатор на състоянието, от което започва преходът.
symbol	Символ, който се използва за прехода. Той трябва да е част от азбуката на автомата или да бъде епсилон.
to	Идентификатор на състоянието, към което води преходът.

4.1.3.2 createState()

```
void Automaton::createNewState (
    size_t id,
    bool isStart,
    bool isFinal)
```

Създава ново състояние с даден идентификатор и го добавя към автомата. Може да се укаже дали състоянието е начално и/или крайно.

Аргументи

id	Идентификатор на новото състояние. Той трябва да е уникален в рамките на автомата.
isStart	Булева стойност, указваща дали новото състояние е начално.
isFinal	Булева стойност, указваща дали новото състояние е крайно.

Изключения

StateExistsException	Ако състоянието с дадения идентификатор вече съществува в автомата, се хвърля изключение от тип StateExistsException .
BadSymbolException	Ако символът, използван в прехода, не е част от азбуката на автомата и не е епсилон, се хвърля изключение от тип BadSymbolException .

4.1.3.3 getAlphabet()

```
const std::vector< char > & Automaton::getAlphabet () const [inline]
```

Връща

азбуката на автомата като вектор от символи.

4.1.3.4 getReversed()

`Automaton` `Automaton::getReversed () const`

Връща нов автомат, който е обърнатата версия на текущия автомат. В обърнатия автомат всички преходи са обърнати, началните състояния стават крайни и крайни състояния стават начални.

Връща

Нов автомат, който е обърнатата версия на текущия автомат.

4.1.3.5 getStatesCount()

`size_t` `Automaton::getStatesCount () const` `[inline]`

Връща

Броя на състоянията в автомата.

4.1.3.6 isDeterministic()

`bool` `Automaton::isDeterministic () const`

Проверява дали автоматът е детерминиран. Автоматът е детерминиран, ако няма две прехода от едно и също състояние с един и същ символ и няма епсилон-преходи.

Връща

`true` - ако автоматът е детерминиран.

4.1.3.7 recognise()

`bool` `Automaton::recognise (`
`const std::string & word) const`

Проверява дали дадена дума се разпознава от автомата.

Аргументи

<code>word</code>	Низ, представляващ думата, която трябва да се провери.
-------------------	--

4.1.4 Приятели и Свързани функции Документация

4.1.4.1 Concat

```
Automaton Concat (  
    const Automaton & lhs,  
    const Automaton & rhs) [friend]
```

Функция за конкатенация на два автомата.

Връща

Нов автомат(Недетерминиран), който представлява конкатенацията на двата входни автомата. Конкатенацията запазва всички състояния и преходи от двата автомата и добавя епсилон-преходи от всички крайни състояния на първия автомат към началните състояния на втория автомат. Новите начални са началните на първия автомат, а новите финални - финалните на втория автомат.

4.1.4.2 Star

```
Automaton Star (  
    const Automaton & automaton) [friend]
```

Функция за звезда на Клини на автомат.

Връща

Нов автомат(Недетерминиран), който представлява звездата на входния автомат. Звездата запазва всички състояния и преходи от входния автомат и добавя ново начално състояние, което е също така крайно. Добавя епсилон-преходи от това ново начално състояние към всички начални състояния на входния автомат, както и епсилон-преходи от всички крайни състояния на входния автомат към това ново начално състояние.

4.1.4.3 Union

```
Automaton Union (  
    const Automaton & lhs,  
    const Automaton & rhs) [friend]
```

Функция за обединение на два автомата.

Връща

Нов автомат(Недетерминиран), който представлява обединението на двата входни автомата. Обединението запазва всички състояния и преходи от двата автомата като новите начални са началните на двата автомата, а новите финални - финалните на двата автомата.

Документация за клас генериран от следните файлове:

- Automaton.hpp
- Automaton.cpp

4.2 AutomatonCLI Клас Препратка

Класът [AutomatonCLI](#) предоставя интерфейс за команден ред, който позволява на потребителите да взаимодействат с автомати и регулярни изрази. Чрез този интерфейс потребителите могат да създават, модифицират, запазват и зареждат автомати, както и да извършват различни операции като детерминизация, минимизация, обединение, конкатенация и звезда. CLI-то също така поддържа разпознаване на думи от автоматите и визуализация на автоматите чрез DOT формат.

```
#include <AutomatonCLI.hpp>
```

Общодостъпни членове функции

- void run ()
- void helpCmd ()
- void exitCmd ()
- void openCmd ()
- void saveCmd ()
- void saveAllCmd ()
- void listCmd ()
- void printCmd ()
- void removeCmd ()
- void resetCmd ()
- void deterministicCmd ()
- void determiniseCmd ()
- void reverseCmd ()
- void minimiseCmd ()
- void recogniseCmd ()
- void unionCmd ()
- void concatCmd ()
- void starCmd ()
- void regCmd ()
- void drawCmd ()

4.2.1 Подробно описание

Класът [AutomatonCLI](#) предоставя интерфейс за команден ред, който позволява на потребителите да взаимодействат с автомати и регулярни изрази. Чрез този интерфейс потребителите могат да създават, модифицират, запазват и зареждат автомати, както и да извършват различни операции като детерминизация, минимизация, обединение, конкатенация и звезда. CLI-то също така поддържа разпознаване на думи от автоматите и визуализация на автоматите чрез DOT формат.

Документация за клас генериран от следните файлове:

- AutomatonCLI.hpp
- AutomatonCLI.cpp

4.3 AutomatonSerializer Клас Препратка

Статични общодостъпни членове функции

- static std::string `serialize` (const `Automaton` &automaton)
Сериализира даден автомат в human-readable формат.
- static `Automaton` `deserialize` (std::istream &is)
Десериализира автомат от human-readable формат. Форматът трябва да съответства на този, описан в `serialize()`.
- static std::string `toDot` (const `Automaton` &automaton)
Генерира представяне на автомата в DOT формат, който може да бъде визуализиран с инструменти като Graphviz. В DOT представянето, състоянията са възли, а преходите са насочени ръбове между тях. Началните състояния са маркирани със зелени стрелки, а крайните състояния са представени с двойни кръгове.
- static void `print` (const `Automaton` &automaton, std::ostream &os=std::cout)

4.3.1 Членове Функции(методи) Документация

4.3.1.1 `serialize()`

```
std::string AutomatonSerializer::serialize (
    const Automaton & automaton) [static]
```

Сериализира даден автомат в human-readable формат.

- Форматът на сериализацията е следният:

```
<брой_символи_в_азбуката> <символи>
<общ_брой_състояния>
<състояние_1> ...
<брой_начални_състояния>
<състояние_1> ...
<брой_финални_състояния>
<състояние_1> ...
<брой_преходи>
<from> <symbol> <to>
...
```

Документация за клас генериран от следните файлове:

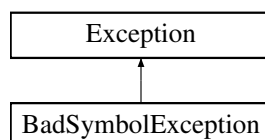
- `AutomatonSerializer.hpp`
- `AutomatonSerializer.cpp`

4.4 BadSymbolException Клас Препратка

Изключение, което се хвърля при опит за използване на символ, който не е част от азбуката на автомата. Съдържа информация за символа, който е бил използван и не е валиден.

```
#include <BadSymbolException.hpp>
```

Диаграма на наследяване за `BadSymbolException`:



Общодостъпни членове функции

- `BadSymbolException (char symbol)`

Общодостъпни членове функции наследен от [Exception](#)

- `Exception (const std::string &message)`
- `virtual std::string toString () const`

4.4.1 Подробно описание

Исключение, което се хвърля при опит за използване на символ, който не е част от азбуката на автомата. Съдържа информация за символа, който е бил използван и не е валиден.

Документация за клас генериран от следният файл:

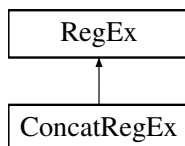
- `BadSymbolException.hpp`

4.5 ConcatRegEx Клас Препратка

Класът [ConcatRegEx](#) е конкатенация на два други регулярни изрази.

```
#include <ConcatRegEx.hpp>
```

Диаграма на наследяване за `ConcatRegEx`:



Общодостъпни членове функции

- `ConcatRegEx (const RegEx &lhs, const RegEx &rhs)`
- `ConcatRegEx (const ConcatRegEx &other)`
- `ConcatRegEx & operator= (const ConcatRegEx &other)`
- `std::string toString () const override`

Връща низово представяне на регулярния израз.

- `Automaton toAutomaton () const override`

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

- `bool eval (const std::string &word) const override`

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

- `RegEx * clone () const override`

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от [RegEx](#)

- static [RegEx](#) * parse (const std::string &expr)
Статична функция, която приема регулярен израз и връща указател към обект от тип [RegEx](#).

4.5.1 Подробно описание

Класът [ConcatRegEx](#) е конкатенация на два други регулярни израза.

4.5.2 Конструктор & Деструктор Документация

4.5.2.1 ConcatRegEx()

```
ConcatRegEx::ConcatRegEx (
    const RegEx & lhs,
    const RegEx & rhs)
```

Аргументи

lhs	Левият израз на конкатенацията, който е указател към обект от тип RegEx .
rhs	Десният израз на конкатенацията, който е указател към обект от тип RegEx .

4.5.3 Членове Функции(методи) Документация

4.5.3.1 clone()

```
RegEx * ConcatRegEx::clone () const [override], [virtual]
```

Заменя [RegEx](#).

4.5.3.2 eval()

```
bool ConcatRegEx::eval (
    const std::string & word) const [override], [virtual]
```

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя [RegEx](#).

4.5.3.3 toAutomaton()

```
Automaton ConcatRegEx::toAutomaton () const [override], [virtual]
```

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя [RegEx](#).

4.5.3.4 toString()

```
std::string ConcatRegEx::toString () const [override], [virtual]
```

Връща низово представяне на регулярния израз.

Заменя [RegEx](#).

Документация за клас генериран от следните файлове:

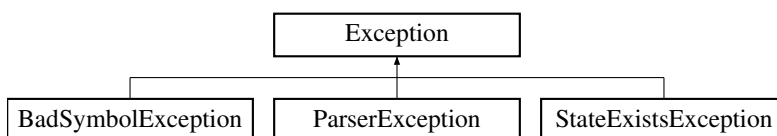
- ConcatRegEx.hpp
- ConcatRegEx.cpp

4.6 Exception Клас Препратка

Базов клас за всички изключения в проекта. Той съдържа съобщение за грешката и предоставя метод `toString()` за връщане на това съобщение.

```
#include <Exception.hpp>
```

Диаграма на наследяване за Exception:



Общодостъпни членове функции

- Exception (const std::string &message)
- virtual std::string toString () const

4.6.1 Подробно описание

Базов клас за всички изключения в проекта. Той съдържа съобщение за грешката и предоставя метод `toString()` за връщане на това съобщение.

Документация за клас генериран от следният файл:

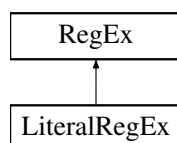
- Exception.hpp

4.7 LiteralRegEx Клас Препратка

Класът [LiteralRegEx](#) представлява регулярни изрази, които съответстват на конкретна дума.

```
#include <LiteralRegEx.hpp>
```

Диаграма на наследяване за LiteralRegEx:



Общодостъпни членове функции

- `LiteralRegEx (const std::string &word)`
Конструктор, който приема дума и създава регулярен израз.
- `bool eval (const std::string &word) const override`
Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.
- `std::string toString () const override`
Връща низово представяне на регулярния израз.
- `Automaton toAutomaton () const override`
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.
- `RegEx * clone () const override`

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от `RegEx`

- `static RegEx * parse (const std::string &expr)`
Статична функция, която приема регулярен израз и връща указател към обект от тип `RegEx`.

4.7.1 Подробно описание

Класът `LiteralRegEx` представлява регулярни изрази, които съответстват на конкретна дума.

4.7.2 Членове Функции(методи) Документация

4.7.2.1 clone()

```
RegEx * LiteralRegEx::clone () const [override], [virtual]
```

Заменя `RegEx`.

4.7.2.2 eval()

```
bool LiteralRegEx::eval (  
    const std::string & word) const [override], [virtual]
```

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя `RegEx`.

4.7.2.3 toAutomaton()

```
Automaton LiteralRegEx::toAutomaton () const [override], [virtual]
```

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя `RegEx`.

4.7.2.4 toString()

```
std::string LiteralRegex::toString () const [override], [virtual]
```

Връща низово представяне на регулярния израз.

Заменя [Regex](#).

Документация за клас генериран от следните файлове:

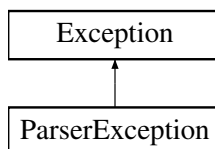
- LiteralRegex.hpp
- LiteralRegex.cpp

4.8 ParserException Клас Препратка

Изключение, което се хвърля при срещане на синтактична грешка по време на парсване на регулярни изрази. Съдържа текст описващ грешката, индекса където е възникнала грешката, и самия израз, който е бил парсван.

```
#include <ParserException.hpp>
```

Диаграма на наследяване за ParserException:



Общодостъпни членове функции

- `ParserException (const std::string &message, const std::string &expr, int idx)`
- `std::string toString () const override`

Общодостъпни членове функции наследен от [Exception](#)

- `Exception (const std::string &message)`

4.8.1 Подробно описание

Изключение, което се хвърля при срещане на синтактична грешка по време на парсване на регулярни изрази. Съдържа текст описващ грешката, индекса където е възникнала грешката, и самия израз, който е бил парсван.

4.8.2 Членове Функции(методи) Документация

4.8.2.1 toString()

`std::string ParseException::toString () const [inline], [override], [virtual]`

Заменя наследеният метод [Exception](#).

Документация за клас генериран от следният файл:

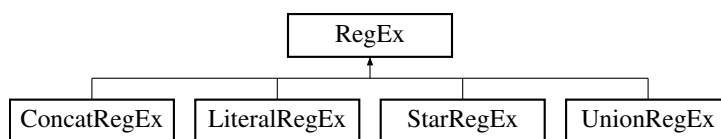
- `ParseException.hpp`

4.9 RegEx Клас Препратка

Абстрактен базов клас за представяне на регулярни изрази.

`#include <RegEx.hpp>`

Диаграма на наследяване за RegEx:



Общодостъпни членове функции

- `virtual bool eval (const std::string &) const =0`
Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.
- `virtual RegEx * clone () const =0`
- `virtual std::string toString () const =0`
Връща низово представяне на регулярния израз.
- `virtual Automaton toAutomaton () const =0`
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Статични общодостъпни членове функции

- `static RegEx * parse (const std::string &expr)`
Статична функция, която приема регулярен израз и връща указател към обект от тип [RegEx](#).

4.9.1 Подробно описание

Абстрактен базов клас за представяне на регулярни изрази.

4.9.2 Членове Функции(методи) Документация

4.9.2.1 eval()

```
virtual bool RegEx::eval (
    const std::string & ) const    [pure virtual]
```

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя в [ConcatRegEx](#), [LiteralRegEx](#), [StarRegEx](#), и [UnionRegEx](#).

4.9.2.2 toAutomaton()

```
virtual Automaton RegEx::toAutomaton () const    [pure virtual]
```

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя в [ConcatRegEx](#), [LiteralRegEx](#), [StarRegEx](#), и [UnionRegEx](#).

4.9.2.3 toString()

```
virtual std::string RegEx::toString () const    [pure virtual]
```

Връща низово представяне на регулярния израз.

Заменя в [ConcatRegEx](#), [LiteralRegEx](#), [StarRegEx](#), и [UnionRegEx](#).

Документация за клас генериран от следните файлове:

- [RegEx.hpp](#)
- [RegEx.cpp](#)

4.10 RegExParser Клас Препратка

Класът [RegExParser](#) използва рекурсивен парсер за обработка на различните оператори в регулярните изрази, като обединение (+), конкатенация(без оператор) и звезда (*). За празна дума се използва символът '@'.

```
#include <RegExParser.hpp>
```

Общодостъпни членове функции

- [RegExParser](#) (const std::string &expr)
Конструктор, който приема регулярен израз.
- [RegEx](#) * parse ()
Функция, която парсва регулярния израз и връща указател към обект от тип [RegEx](#), който представлява този израз.

4.10.1 Подробно описание

Класът [RegExParser](#) използва рекурсивен парсер за обработка на различните оператори в регулярните изрази, като обединение (+), конкатенация(без оператор) и звезда (*). За празна дума се използва символът '@'.

4.10.2 Членове Функции(методи) Документация

4.10.2.1 parse()

[RegEx](#) * RegExParser::parse ()

Функция, която парсва регулярния израз и връща указател към обект от тип [RegEx](#), който представлява този израз.

Изключения

ParserException	Ако по време на парсването се срещне синтактична грешка, се хвърля изключение от тип ParserException , което съдържа информация за грешката и позицията в израза, където е възникнала.
-----------------	--

Документация за клас генериран от следните файлове:

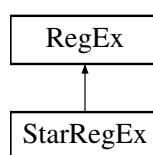
- RegExParser.hpp
- RegExParser.cpp

4.11 StarRegEx Клас Препратка

Класът [StarRegEx](#) представлява регулярни изрази, които са звезда на друг регулярен израз.

```
#include <StarRegEx.hpp>
```

Диаграма на наследяване за StarRegEx:



Общодостъпни членове функции

- [StarRegEx](#) (const [RegEx](#) &inner)
Конструктор, който приема друг регулярен израз и създава звезда на този израз.
- [StarRegEx](#) (const [StarRegEx](#) &other)
- [StarRegEx](#) & operator= (const [StarRegEx](#) &other)
- bool [eval](#) (const std::string &word) const override
Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.
- [Automaton toAutomaton](#) () const override
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.
- std::string [toString](#) () const override
Връща низово представяне на регулярния израз.
- [RegEx](#) * [clone](#) () const override

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от [RegEx](#)

- static [RegEx](#) * parse (const std::string &expr)

Статична функция, която приема регулярен израз и връща указател към обект от тип [RegEx](#).

4.11.1 Подробно описание

Класът [StarRegEx](#) представлява регулярни изрази, които са звезда на друг регулярен израз.

4.11.2 Членове Функции(методи) Документация

4.11.2.1 clone()

[RegEx](#) * StarRegEx::clone () const [override], [virtual]

Заменя [RegEx](#).

4.11.2.2 eval()

bool StarRegEx::eval (
const std::string & word) const [override], [virtual]

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя [RegEx](#).

4.11.2.3 toAutomaton()

[Automaton](#) StarRegEx::toAutomaton () const [override], [virtual]

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя [RegEx](#).

4.11.2.4 toString()

std::string StarRegEx::toString () const [override], [virtual]

Връща низово представяне на регулярния израз.

Заменя [RegEx](#).

Документация за клас генериран от следните файлове:

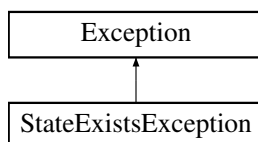
- StarRegEx.hpp
- StarRegEx.cpp

4.12 StateExistsException Клас Препратка

Изключение, което се хвърля при опит за създаване на състояние с идентификатор, който вече съществува в автомата, или при опит за достъп до несъществуващо състояние. Съдържа информация за идентификатора на състоянието и дали то вече съществува или не.

```
#include <StateExistsException.hpp>
```

Диаграма на наследяване за StateExistsException:



Общодостъпни членове функции

- StateExistsException (size_t id, bool exists)
- std::string toString () const override
- bool exists () const
- size_t id () const

Общодостъпни членове функции наследен от [Exception](#)

- Exception (const std::string &message)

4.12.1 Подробно описание

Изключение, което се хвърля при опит за създаване на състояние с идентификатор, който вече съществува в автомата, или при опит за достъп до несъществуващо състояние. Съдържа информация за идентификатора на състоянието и дали то вече съществува или не.

4.12.2 Членове Функции(методи) Документация

4.12.2.1 toString()

```
std::string StateExistsException::toString () const [inline], [override], [virtual]
```

Заменя наследеният метод [Exception](#).

Документация за клас генериран от следният файл:

- StateExistsException.hpp

4.13 Transition Структура Препратка

Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като '\0').

```
#include <Automaton.hpp>
```

Общодостъпни атрибути

- size_t from
- char symbol
- size_t to

4.13.1 Подробно описание

Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като '\0').

Документация за структура генериран от следният файл:

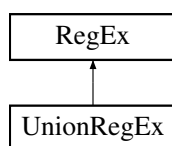
- Automaton.hpp

4.14 UnionRegEx Клас Препратка

Класът [UnionRegEx](#) представлява обединение на два други регулярни изрази.

```
#include <UnionRegEx.hpp>
```

Диаграма на наследяване за UnionRegEx:



Общодостъпни членове функции

- [UnionRegEx](#) (const [RegEx](#) &lhs, const [RegEx](#) &rhs)
- [UnionRegEx](#) (const [UnionRegEx](#) &other)
- [UnionRegEx](#) & operator= (const [UnionRegEx](#) &other)
- std::string [toString](#) () const override

Връща низово представяне на регулярния израз.

- [Automaton toAutomaton](#) () const override

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

- bool [eval](#) (const std::string &word) const override

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

- [RegEx * clone](#) () const override

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от [RegEx](#)

- static [RegEx](#) * parse (const std::string &expr)
Статична функция, която приема регулярен израз и връща указател към обект от тип [RegEx](#).

4.14.1 Подробно описание

Класът [UnionRegEx](#) представлява обединение на два други регулярни изрази.

4.14.2 Конструктор & Деструктор Документация

4.14.2.1 UnionRegEx()

```
UnionRegEx::UnionRegEx (
    const RegEx & lhs,
    const RegEx & rhs)
```

Аргументи

lhs	Левият израз на обединението, който е указател към обект от тип RegEx .
rhs	Десният израз на обединението, който е указател към обект от тип RegEx .

4.14.3 Членове Функции(методи) Документация

4.14.3.1 clone()

```
RegEx * UnionRegEx::clone () const [override], [virtual]
```

Заменя [RegEx](#).

4.14.3.2 eval()

```
bool UnionRegEx::eval (
    const std::string & word) const [override], [virtual]
```

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя [RegEx](#).

4.14.3.3 toAutomaton()

```
Automaton UnionRegEx::toAutomaton () const [override], [virtual]
```

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя [RegEx](#).

4.14.3.4 toString()

`std::string UnionRegex::toString () const [override], [virtual]`

Връща низово представяне на регулярния израз.

Заменя [Regex](#).

Документация за клас генериран от следните файлове:

- UnionRegex.hpp
- UnionRegex.cpp

Глава 5

Файлове Документация

5.1 Automaton.hpp

```
00001 #pragma once
00002 #include <map>
00003 #include <vector>
00004 #include <string>
00005 #include <set>
00006 #include <iostream>
00007
00008 #include "../Exceptions/Exception.hpp"
00009 #include "../Exceptions/StateExistsException.hpp"
00010 #include "../Exceptions/BadSymbolException.hpp"
00011
00012 constexpr char EPSILON = '\0';
00013
00014
00015 struct Transition
00016 {
00017     size_t from;
00018     char symbol;
00019     size_t to;
00020 };
00021
00022 class Automaton
00023 {
00024 public:
00025     friend class AutomatonSerializer;
00026
00027     Automaton(const std::vector<char> alphabet);
00028
00029     void createState(size_t id, bool isStart, bool isFinal);
00030
00031     void addTransition(size_t from, char symbol, size_t to);
00032
00033     bool recognise(const std::string& word) const;
00034
00035     Automaton getReversed() const;
00036     void reverse();
00037
00038     bool isDeterministic() const;
00039     Automaton getDeterministic() const;
00040
00041     void determinise();
00042
00043     Automaton getMinimised() const;
00044
00045     void minimise();
00046
00047     friend Automaton Union(const Automaton& lhs, const Automaton& rhs);
00048
00049     friend Automaton Concat(const Automaton& lhs, const Automaton& rhs);
00050
00051     friend Automaton Star(const Automaton& automaton);
00052
00053     const std::vector<char> getAlphabet() const
00054     {
00055         return m_alphabet;
00056     }
00057 }
```

```

00124     ~Automaton() = default;
00125
00129     size_t getStatesCount() const
00130     {
00131         return m_states.size();
00132     }
00133
00134 private:
00135     bool hasState(size_t id) const
00136     {
00137         for (size_t state : m_states)
00138             if (state == id) return true;
00139         return false;
00140     }
00141     bool hasAnyTransitions(size_t state) const
00142     {
00143         for (const Transition& transition : m_transitions)
00144             if (transition.from == state) return true;
00145         return false;
00146     }
00147     bool symbolsInAlphabet(char symbol) const
00148     {
00149         for (char s : m_alphabet)
00150         {
00151             if (s == symbol) return true;
00152         }
00153         return false;
00154     }
00155
00156     std::vector<size_t> bubbleSort(const std::vector<size_t>& arr) const;
00157
00158     void pushReachableSubsets(Automaton& dfa, std::vector<std::vector<size_t>& stateSubsets) const;
00159
00160     void addSubsetTransitions(Automaton& dfa, const std::vector<std::vector<size_t>& stateSubsets) const;
00161
00162
00163     void pushStatesWithEpsilon(std::vector<size_t>& closure, std::vector<size_t>& stack, size_t stateId) const;
00164
00165     std::vector<size_t> getEpsilonClosure(const std::vector<size_t>& states) const;
00166     std::vector<size_t> getNextStates(const std::vector<size_t>& currentStates, char symbol) const;
00167
00168     bool containsFinalState(const std::vector<size_t>& currentStates) const;
00169 private:
00170     std::vector<size_t> m_states;
00171     std::vector<size_t> m_startStates;
00172     std::vector<size_t> m_finalStates;
00173
00174     std::vector<Transition> m_transitions;
00175     std::vector<char> m_alphabet;
00176 };

```

5.2 AutomatonSerializer.hpp

```

00001 #pragma once
00002 #include <string>
00003 #include <vector>
00004 #include <iostream>
00005 #include "Automaton.hpp"
00006
00007 class AutomatonSerializer
00008 {
00009 public:
00010
00027     static std::string serialize(const Automaton& automaton);
00028
00032     static Automaton deserialize(std::istream& is);
00033
00037     static std::string toDot(const Automaton& automaton);
00038
00039     static void print(const Automaton& automaton, std::ostream& os = std::cout);
00040
00041 private:
00042     static std::string serializeStates(const std::vector<size_t>& container);
00043
00044     static std::string serializeTransitions(const Automaton& automaton);
00045
00046     static Automaton deserializeAlphabet(std::istream& is);
00047
00048     static void deserializeStates(Automaton& automaton, std::istream& is);
00049
00050     static void deserializeTransitions(Automaton& automaton, std::istream& is);
00051
00052     AutomatonSerializer() = default;
00053 };

```

5.3 BadSymbolException.hpp

```

00001 #include "Exception.hpp"
00002
00003
00007 class BadSymbolException : public Exception
00008 {
00009     char m_symbol;
00010 public:
00011     BadSymbolException(char symbol) : Exception("Symbol '" + std::string(1, symbol) + "' is not in the alphabet."),
        m_symbol(symbol) {}
00012     virtual ~BadSymbolException() = default;
00013 };

```

5.4 Exception.hpp

```

00001 #pragma once
00002 #include <string>
00003
00004
00008 class Exception
00009 {
00010     const std::string m_message;
00011 public:
00012     Exception(const std::string& message) : m_message(message) {}
00013     virtual std::string toString() const { return m_message; }
00014     virtual ~Exception() = default;
00015 };

```

5.5 ParserException.hpp

```

00001 #include "Exception.hpp"
00002
00003
00007 class ParserException : public Exception
00008 {
00009     int idx;
00010     std::string expr;
00011 public:
00012     ParserException(const std::string& message, const std::string& expr, int idx) : Exception(message), expr(expr), idx(idx)
        {}
00013     std::string toString() const override
00014     {
00015         return Exception::toString() + " at index " + std::to_string(idx) + " in expression: " + expr;
00016     }
00017 };

```

5.6 StateExistsException.hpp

```

00001 #pragma once
00002 #include "Exception.hpp"
00003
00004
00005
00009 class StateExistsException : public Exception
00010 {
00011     size_t m_id;
00012     bool m_exists;
00013 public:
00014     StateExistsException(size_t id, bool exists) : Exception(""), m_id(id), m_exists(exists) {}
00015     std::string toString() const override
00016     {
00017         return "State " + std::to_string(m_id) + (m_exists ? " already exists." : " does not exist.");
00018     }
00019
00020     bool exists() const { return m_exists; }
00021     size_t id() const { return m_id; }
00022     virtual ~StateExistsException() = default;
00023 };

```

5.7 ConcatRegEx.hpp

```

00001 #pragma once
00002 #include "RegEx.hpp"
00003
00004
00008 class ConcatRegEx : public RegEx
00009 {
00010 public:
00011
00016     ConcatRegEx(const RegEx& lhs, const RegEx& rhs);
00017     ConcatRegEx(const ConcatRegEx& other);
00018     ConcatRegEx& operator=(const ConcatRegEx& other);
00019     virtual ~ConcatRegEx();
00020
00021
00022     std::string toString() const override;
00023     Automaton toAutomaton() const override;
00024     bool eval(const std::string& word) const override;
00025     RegEx* clone() const override;
00026
00027 private:
00028     RegEx* lhs;
00029     RegEx* rhs;
00030     void free();
00031 };

```

5.8 LiteralRegEx.hpp

```

00001 #pragma once
00002 #include "RegEx.hpp"
00003
00004
00008 class LiteralRegEx : public RegEx
00009 {
00010 public:
00011
00015     LiteralRegEx(const std::string& word);
00016
00017     bool eval(const std::string& word) const override;
00018
00019     std::string toString() const override;
00020
00021     Automaton toAutomaton() const override;
00022
00023     RegEx* clone() const override;
00024 private:
00025     std::string m_word;
00026 };

```

5.9 RegEx.hpp

```

00001 #pragma once
00002 #include <string>
00003 #include "../Automaton/Automaton.hpp"
00004
00008 class RegEx
00009 {
00010 public:
00011
00015     virtual bool eval(const std::string&) const = 0;
00016     virtual RegEx* clone() const = 0;
00017
00021     virtual std::string toString() const = 0;
00022
00026     virtual Automaton toAutomaton() const = 0;
00027
00028     virtual ~RegEx() = default;
00029
00033     static RegEx* parse(const std::string& expr);
00034 };

```

5.10 RegExParser.hpp

```

00001 #pragma once

```

```

00002 #include "RegEx.hpp"
00003 #include "StarRegEx.hpp"
00004 #include "UnionRegEx.hpp"
00005 #include "ConcatRegEx.hpp"
00006 #include "LiteralRegEx.hpp"
00007
00008
00012 class RegExParser
00013 {
00014 public:
00018     RegExParser(const std::string& expr);
00019
00024     RegEx* parse();
00025 private:
00026     RegEx* parseUnion();
00027     RegEx* parseConcat();
00028     RegEx* parseStar();
00029     RegEx* parseBrackets();
00030     RegEx* parseLiteral();
00031     std::string m_expr;
00032     size_t m_idx;
00033     char peek() const;
00034     char get();
00035 };

```

5.11 StarRegEx.hpp

```

00001 #pragma once
00002 #include "RegEx.hpp"
00003
00004
00008 class StarRegEx : public RegEx
00009 {
00010 public:
00011
00015     StarRegEx(const RegEx& inner);
00016     StarRegEx(const StarRegEx& other);
00017     StarRegEx& operator=(const StarRegEx& other);
00018     virtual ~StarRegEx();
00019     bool eval(const std::string& word) const override;
00020     Automaton toAutomaton() const override;
00021     std::string toString() const override;
00022     RegEx* clone() const override;
00023 private:
00024     RegEx* inner;
00025     void free();
00026 };

```

5.12 UnionRegEx.hpp

```

00001 #pragma once
00002 #include "RegEx.hpp"
00003
00004
00008 class UnionRegEx : public RegEx
00009 {
00010 public:
00011
00016     UnionRegEx(const RegEx& lhs, const RegEx& rhs);
00017     UnionRegEx(const UnionRegEx& other);
00018     UnionRegEx& operator=(const UnionRegEx& other);
00019     virtual ~UnionRegEx();
00020     std::string toString() const override;
00021     Automaton toAutomaton() const override;
00022     bool eval(const std::string& word) const override;
00023     RegEx* clone() const override;
00024
00025 private:
00026     RegEx* lhs;
00027     RegEx* rhs;
00028     void free();
00029 };

```

5.13 AutomatonCLI.hpp

```

00001 #pragma once

```

```

00002 #include <iostream>
00003 #include <fstream>
00004 #include <string>
00005
00006 #include "../Automaton/Automaton.hpp"
00007 #include "../RegEx/UnionRegEx.hpp"
00008 #include "../RegEx/ConcatRegEx.hpp"
00009 #include "../RegEx/StarRegEx.hpp"
00010 #include "../RegEx/LiteralRegEx.hpp"
00011 #include "../RegEx/RegExParser.hpp"
00012 #include "../Automaton/AutomatonSerializer.hpp"
00013
00014
00015
00019 class AutomatonCLI
00020 {
00021     struct IdAutomaton
00022     {
00023         size_t id;
00024         Automaton automaton;
00025     };
00026     std::vector<IdAutomaton> m_automatons;
00027     Automaton& getAutomatonById(size_t id);
00028     void removeAutomatonById(size_t id);
00029     void addAutomaton(const Automaton& automaton);
00030     bool m_isRunning = true;
00031     size_t idCounter = 0;
00032     size_t getNextId();
00033     void reset();
00034 public:
00035     void run();
00036     void helpCmd();
00037     void exitCmd();
00038     void openCmd();
00039     void saveCmd();
00040     void saveAllCmd();
00041     void listCmd();
00042     void printCmd();
00043     void removeCmd();
00044     void resetCmd();
00045     void deterministicCmd();
00046     void determiniseCmd();
00047     void reverseCmd();
00048     void minimiseCmd();
00049     void recogniseCmd();
00050     void unionCmd();
00051     void concatCmd();
00052     void starCmd();
00053     void regCmd();
00054     void drawCmd();
00055 };

```

Азбучен указател

- addTransition
 - Automaton, [9](#)
- Automaton, [7](#)
 - addTransition, [9](#)
 - Automaton, [8](#)
 - Concat, [11](#)
 - createNewState, [9](#)
 - getAlphabet, [9](#)
 - getReversed, [9](#)
 - getStatesCount, [10](#)
 - isDeterministic, [10](#)
 - recognise, [10](#)
 - Star, [11](#)
 - Union, [11](#)
- Automaton.hpp, [27](#)
- AutomatonCLI, [12](#)
- AutomatonCLI.hpp, [31](#)
- AutomatonSerializer, [13](#)
 - serialize, [13](#)
- AutomatonSerializer.hpp, [28](#)
- BadSymbolException, [13](#)
- BadSymbolException.hpp, [29](#)
- clone
 - ConcatRegEx, [15](#)
 - LiteralRegEx, [17](#)
 - StarRegEx, [22](#)
 - UnionRegEx, [25](#)
- Concat
 - Automaton, [11](#)
- ConcatRegEx, [14](#)
 - clone, [15](#)
 - ConcatRegEx, [15](#)
 - eval, [15](#)
 - toAutomaton, [15](#)
 - toString, [15](#)
- ConcatRegEx.hpp, [30](#)
- createNewState
 - Automaton, [9](#)
- eval
 - ConcatRegEx, [15](#)
 - LiteralRegEx, [17](#)
 - RegEx, [20](#)
 - StarRegEx, [22](#)
 - UnionRegEx, [25](#)
- Exception, [16](#)
- Exception.hpp, [29](#)
- getAlphabet
 - Automaton, [9](#)
- getReversed
 - Automaton, [9](#)
- getStatesCount
 - Automaton, [10](#)
- isDeterministic
 - Automaton, [10](#)
- LiteralRegEx, [16](#)
 - clone, [17](#)
 - eval, [17](#)
 - toAutomaton, [17](#)
 - toString, [17](#)
- LiteralRegEx.hpp, [30](#)
- parse
 - RegExParser, [21](#)
- ParserException, [18](#)
 - toString, [19](#)
- ParserException.hpp, [29](#)
- recognise
 - Automaton, [10](#)
- RegEx, [19](#)
 - eval, [20](#)
 - toAutomaton, [20](#)
 - toString, [20](#)
- RegEx.hpp, [30](#)
- RegExParser, [20](#)
 - parse, [21](#)
- RegExParser.hpp, [30](#)
- serialize
 - AutomatonSerializer, [13](#)
- Star
 - Automaton, [11](#)
- StarRegEx, [21](#)
 - clone, [22](#)
 - eval, [22](#)
 - toAutomaton, [22](#)
 - toString, [22](#)
- StarRegEx.hpp, [31](#)
- StateExistsException, [23](#)
 - toString, [23](#)
- StateExistsException.hpp, [29](#)
- toAutomaton
 - ConcatRegEx, [15](#)

- LiteralRegEx, [17](#)
- RegEx, [20](#)
- StarRegEx, [22](#)
- UnionRegEx, [25](#)
- toString
 - ConcatRegEx, [15](#)
 - LiteralRegEx, [17](#)
 - ParserException, [19](#)
 - RegEx, [20](#)
 - StarRegEx, [22](#)
 - StateExistsException, [23](#)
 - UnionRegEx, [25](#)
- Transition, [24](#)
- Union
 - Automaton, [11](#)
- UnionRegEx, [24](#)
 - clone, [25](#)
 - eval, [25](#)
 - toAutomaton, [25](#)
 - toString, [25](#)
 - UnionRegEx, [25](#)
- UnionRegEx.hpp, [31](#)